

WEEK 2 · NOTES &
ASSIGNMENT

Conditionals.

How to make decisions in code.

The 4 conditional shapes every developer reaches for — and how to pick the right one before you type a single `if`.

CORE SKILL

**Map Decisions Before
Code**

FRAMEWORK

The 4 Conditional Shapes

ASSIGNMENT

10 Decision Problems

PART 01 · THE MINDSET

From Building to Deciding

Last week, you learned how to break any problem into **Inputs** → **Process** → **Output** → **Edges**. This week we zoom into one specific part of that — the moment in **PROCESS** where your code has to **make a choice**.

You already wrote `if/else` last week. But you wrote it however felt natural. The code worked, but it was messy: nested 3 levels deep, conditions in random order, the same check written twice. That's not your fault — nobody taught you the shapes. **Today they will be obvious to you.**

Two Students. Same Problem. Different Code.

STUDENT A · CODES BY FEEL*Messy. Buggy. Hard to read.*

```
if (price > 500) {  
  if (member) {  
    if (day === "Saturday") {  
      // ... 3 levels deep  
    }  
  }  
}  
} else if (price > 500) {  
  // wait, did I check this already?  
}
```

*Spends 2 hours debugging.***STUDENT B · MAPS THE TREE FIRST***Clean. Predictable. Easy to extend.*

```
// 1. Is the input valid?  
// 2. Pick the right branch.  
// 3. Apply discount.  
// 4. Show result.  
//  
// Tree drawn on paper.  
// Then 12 clean lines.
```

*Done in 15 minutes. Zero bugs.***THE TRUTH NOBODY TOLD YOU**

Bad conditionals are the **#1 source of bugs** in beginner code. Not syntax errors. Not missing semicolons. **Wrong order. Missing else. Repeated checks.** Today you learn to spot all three.

PART 02 · THE FRAMEWORK

The 4 Conditional Shapes

Every decision in code fits into one of **four shapes**. Most beginners only know one (if/else) and force every problem into it. Once you see the four, you'll pick the right one automatically.

01

SINGLE IF/ELSE

Binary choice

*Member vs non-member,
valid vs invalid input*

02

ELSE-IF CHAIN

Ordered ranges

*Tax slabs, BMI bands, age
groups*

03

SWITCH

Discrete known values

*Day of week, payment
method*

04

LOOKUP OBJECT

1:1 mapping

*Seat type → price, color →
action*

Side by Side: switch vs Lookup Object

Same problem (traffic light → action). Two valid solutions. Compare them.

LOOKUP OBJECT · CLEANER, SHORTER

```
const actions = {
  red: "Stop",
  yellow: "Slow down",
  green: "Go"
};

if (actions[color]) {
  console.log(actions[color]);
} else {
  console.log("Invalid signal");
}
```

SWITCH · MORE LINES, BUT EXPLICIT

```
switch (color) {
  case "red":
    console.log("Stop");
    break;
  case "yellow":
    console.log("Slow down");
    break;
  case "green":
    console.log("Go");
}
```

```
    break;  
  default:  
    console.log("Invalid signal");  
}
```

MEMORIZE THIS RULE

Reach for a lookup object 90% of the time. Reach for switch only when each case does something different — not just returns a different value.

PART 03 · THE WALKTHROUGH

Movie Ticket Pricing v2

This is what "choosing the right shape" actually looks like. We'll build the pricing logic for a real multiplex — with seat types, weekend surcharge, group discount, and member tiers. **All 4 conditional shapes appear.**

Stage A · The Requirements

Before you write a single line — read the brief. This is what a real client would hand you:

CLIENT BRIEF · MULTIPLEX TICKET CALCULATOR

Build a ticket pricing calculator. The customer picks a **day, seat type, number of tickets**, and **membership tier**. The system calculates the final price. **Seat prices:** Regular ₹200, Premium ₹350, Recliner ₹500 **Weekend surcharge:** +₹50 per ticket on Saturday and Sunday **Group discount:** 10% off subtotal if 4 or more tickets **Member discount:** Platinum 15%, Gold 10%, Silver 5%, None 0% **Reject:** zero/negative tickets, invalid seat types, invalid days.

Stage B · Apply the Framework

Notebook open, VS Code closed. Talk through it out loud.

PLAIN-ENGLISH DECOMPOSITION

PROBLEM: Calculate movie ticket price.

INPUTS:

- Day (e.g. "Saturday")
- Seat type (Regular / Premium / Recliner)
- Number of tickets
- Member tier (Platinum / Gold / Silver / None)

PROCESS:

- Look up base seat price
- Add weekend surcharge if Sat/Sun
- Multiply by ticket count → subtotal
- Apply 10% group discount if tickets ≥ 4
- Apply member discount based on tier
- That's your final total

OUTPUT:

- Price per ticket (after weekend surcharge)

- Subtotal, group discount, member discount
- Final total

EDGES:

- numTickets is 0 or negative → reject
- Seat type not in {Regular, Premium, Recliner} → reject
- Member tier "None" or invalid → no discount, don't crash

Stage C · Translate to Pseudocode

Closer to code, still no real JavaScript. Each line is one step.

PSEUDOCODE

GET day, seatType, numTickets, memberTier from user

DEFINE seatPrices:

Regular → 200

Premium → 350

Recliner → 500

DEFINE memberDiscount:

Platinum → 0.15

Gold → 0.10

Silver → 0.05

None → 0

IF numTickets <= 0:

SHOW "Invalid ticket count"

STOP

IF seatType not in seatPrices:

SHOW "Invalid seat type"

STOP

SET pricePerTicket = seatPrices[seatType]

IF day is "Saturday" OR "Sunday":

ADD 50 to pricePerTicket

CALCULATE subtotal = pricePerTicket * numTickets

IF numTickets >= 4:

APPLY 10% group discount to subtotal

```
discountRate = memberDiscount[memberTier] OR 0
APPLY discountRate to subtotal → final

SHOW pricePerTicket, subtotal, discounts, final
```

Stage D · Now Open VS Code

Each pseudocode block maps to a few JavaScript lines. The code writes itself.

```
MOVIE-TICKET.JS

// Inputs from user
let day = prompt("Enter day (e.g., Saturday):");
let seatType = prompt("Seat type (Regular / Premium / Recliner):");
let numTickets = Number(prompt("How many tickets?"));
let memberTier = prompt("Member tier (Platinum / Gold / Silver / None):");

// Lookup tables – pick the right price/discount in one step
const seatPrices = {
  Regular: 200,
  Premium: 350,
  Recliner: 500
};

const memberDiscount = {
  Platinum: 0.15,
  Gold: 0.10,
  Silver: 0.05,
  None: 0
};

// Validation – fail fast, succeed last
if (numTickets <= 0 || isNaN(numTickets)) {
  console.log("Invalid ticket count");
} else if (seatPrices[seatType] === undefined) {
  console.log("Invalid seat type");
} else {

  // Step 1: base price + weekend surcharge
  let pricePerTicket = seatPrices[seatType];
  if (day === "Saturday" || day === "Sunday") {
    pricePerTicket = pricePerTicket + 50;
  }

  // Step 2: subtotal
```

```
let subtotal = pricePerTicket * numTickets;

// Step 3: group discount if 4+ tickets
let groupDiscount = 0;
if (numTickets >= 4) {
  groupDiscount = subtotal * 0.10;
  subtotal = subtotal - groupDiscount;
}

// Step 4: member discount
let rate = memberDiscount[memberTier] || 0;
let memberAmount = subtotal * rate;
let finalTotal = subtotal - memberAmount;

// Step 5: show breakdown
console.log("Price per ticket: ₹" + pricePerTicket);
console.log("Tickets:           " + numTickets);
console.log("Group discount:    ₹" + groupDiscount);
console.log("Member discount: ₹" + memberAmount.toFixed(2));
console.log("FINAL TOTAL:      ₹" + finalTotal.toFixed(2));
}
```

Sample Run

Input: Saturday, Premium, 4, Gold

OUTPUT

Price per ticket: ₹400	← (350 base + 50 weekend)
Tickets: 4	
Group discount: ₹160	← 10% off (4+ tickets)
Member discount: ₹144.00	← 10% off the discounted subtotal
FINAL TOTAL: ₹1296.00	

THE LIGHTBULB MOMENT

Look at the code. Four conditional shapes — single `if` (validation), lookup object (seat prices), combined condition (group), lookup with fallback (member). **All in one realistic problem.** This is what "choosing the right shape" looks like.

PART 04 · THIS WEEK'S WORK

Week 2 · Exercise Pack

10 problems. Pseudocode is mandatory for every problem. JavaScript is mandatory this week (you've earned it). Each problem includes EDGES — you must handle them, not skip them.

THE RULE

If you write JavaScript before pseudocode, your answer is wrong — even if it works. The whole point is to *map decisions before you write them*. Habit > output, every single time.

01 The Even / Odd Checker

★ EASY

Take a number from the user. Tell them whether it's Even, Odd, or Zero.

EDGES zero (handle separately), negative numbers, non-numeric input

02 The Login Validator

★ EASY

Ask for username and password. The correct username is `rajdeep` and the correct password is `easy123`. Print one of: "Login successful," "Wrong password," "User not found," or "Both fields empty."

EDGES both empty (one combined message), correct user but wrong password, user doesn't exist

03 The BMI Classifier

★ MEDIUM

Take weight (kg) and height (m). Calculate $BMI = \text{weight} \div (\text{height} \times \text{height})$. Classify as:

- Below 18.5 → "Underweight"
- 18.5 – 24.9 → "Normal"
- 25 – 29.9 → "Overweight"
- 30 and above → "Obese"

EDGES height = 0 (would crash division), negative inputs, decimal inputs

04 The Pizza Order Validator

★ MEDIUM

Build the order validator for a pizza app. Take size, toppings count, and delivery type.

- **Sizes:** Small ₹199, Medium ₹299, Large ₹399
- **Toppings:** ₹40 per topping (max 5 toppings)
- **Delivery:** "pickup" (free) or "delivery" (₹40, free over ₹500)
- **Output:** base price, toppings cost, delivery fee, final total

EDGES invalid size, more than 5 toppings, negative toppings count, invalid delivery type

05 The Traffic Light Action

★ MEDIUM

User enters a color: red, yellow, or green. Print "Stop," "Slow down," or "Go." Anything else → "Invalid signal."

Solve this twice — once with switch, once with a lookup object. Notice which one feels cleaner.

EDGES capitalization ("RED" vs "red"), empty string, numbers as input

06 The Tiered Discount Calculator

★ MEDIUM

Take cart total. Apply discount based on tier:

- Below ₹500 → No discount
- ₹500 – ₹1,999 → 5% off
- ₹2,000 – ₹4,999 → 10% off
- ₹5,000 and above → 15% off

Output: original total, discount amount, final amount.

EDGES boundary values (₹500, ₹2000, ₹5000), negative total, zero

07 The Day Activity Recommender

★ MEDIUM

Take the day of the week. Recommend an activity:

- Monday → "Plan the week"
- Tuesday / Wednesday → "Deep work day"
- Thursday → "Review progress"
- Friday → "Wrap up"
- Saturday / Sunday → "Rest or hobby"

Solve using a lookup object — even though some days share output. Figure out how.

EDGES invalid day name, capitalization mismatch, empty input

08 The Income Tax Calculator

★
CHALLENGE

Calculate Indian income tax (simplified slabs):

- Up to ₹2,50,000 → No tax
- ₹2,50,001 – ₹5,00,000 → 5% on amount above ₹2,50,000
- ₹5,00,001 – ₹10,00,000 → ₹12,500 + 20% on amount above ₹5,00,000
- Above ₹10,00,000 → ₹1,12,500 + 30% on amount above ₹10,00,000

Output: total income, tax payable, take-home amount.

EDGES boundary income (₹5,00,000), negative income, zero income

09

The Vending Machine ★ STAR PROBLEM

★
C H A L L E N G
E

This is the deepest problem of the week. Get this right and conditionals stop being scary forever.

Build the logic for a vending machine.

- **Items:** Chips ₹20, Soda ₹35, Chocolate ₹50, Water ₹15
- **Stock:** Chips (3), Soda (0), Chocolate (5), Water (2)
- **User enters:** item name + amount of money

Your code must:

- 1. Check if the item exists
- 2. Check if it's in stock
- 3. Check if money is enough
- 4. If all good — dispense, calculate change, reduce stock
- 5. If anything fails — return money with the right error message

EDGES item not in menu, item out of stock (Soda), insufficient money, exact change, ₹0 or negative inserted

10

Rock Paper Scissors with Score

★
C H A L L E N G
E

User plays 3 rounds against the computer. Each round:

- User picks rock, paper, or scissors
- Computer picks randomly
- Determine round winner, track score

After round 3, declare the overall winner.

EDGES tie in a round (no one scores), final tie across 3 rounds, invalid input like "rcok"

BONUS · OPTIONAL

Decompose a Real App

Pick **any decision-making feature** in an app you used today. Examples: Zomato deciding whether to charge a delivery fee. Uber deciding surge pricing. WhatsApp deciding whether a message is delivered, read, or pending. ChatGPT deciding whether your prompt needs a long answer or short one.

Apply the framework. **Don't code it.** Just decompose it: identify the inputs, the decision tree, the output, and the edge cases. Post your decomposition in the WhatsApp group. Best one gets a shoutout in Friday's Logic Lab.

PART 05 · THE PROCESS

How to Submit

SUBMISSION RULE

This week we're stepping it up. **Pseudocode AND JavaScript** are both required for every problem. The thinking habit was Week 1 — now we ship working code that comes from clean thinking.

Folder Structure

Organize your submissions like a real developer would. One folder per problem, all 10 inside one main folder, then zipped.

FOLDER LAYOUT

```
week-2-firstname/  
├─ problem-01-even-odd/  
│   ├── pseudocode.txt  
│   └─ solution.js  
├─ problem-02-login-validator/  
│   ├── pseudocode.txt  
│   └─ solution.js  
├─ problem-03-bmi/  
│   ├── pseudocode.txt  
│   └─ solution.js  
├─ ... (problem 04 to 10)  
└─ bonus-decompose/  
    └─ decomposition.txt
```

Then zip the entire week-2-firstname folder and upload.

Naming Convention

Replace `firstname` with your actual first name (lowercase, no spaces). For example: `week-2-rajdeep`, `week-2-priya`, `week-2-arjun`.

Why this matters: real engineering teams care about clean naming. Sloppy file names = sloppy submissions. We grade on the thinking, but we notice the craft.

Upload Link

Drop your zipped folder here: [\[upload link will be shared in the WhatsApp group\]](#)

The link gets posted in the group on Sunday morning along with the recording. If you can't find it, ping @Rajdeep — don't email.

Submission Checklist

- ☐ **Pseudocode for all 10 problems** — INPUTS, PROCESS, OUTPUT, EDGES + steps in plain English
- ☐ **JavaScript for all 10 problems** — clean, runnable, mirrors the pseudocode
- ☐ **Folder structure correct** — one folder per problem, named `problem-01-...` through `problem-10-...`
- ☐ **Main folder named correctly** — `week-2-firstname` (lowercase)
- ☐ **Zipped before upload** — no loose files, no `.DS_Store`, no `node_modules`
- ☐ **Bonus decomposition (optional)** — in its own folder if you did it

BEFORE YOU UPLOAD — SANITY CHECK

Open the zip in a different folder. Can you actually run `solution.js` for each problem? Does the pseudocode actually match what the code does? **If yes, you're ready. If no, fix it before submitting.** Sloppy submissions get a thumbs-down review and you'll have to resubmit.

CLOSING THOUGHT

Last week you learned how to **think**. This week you learn how to **decide**. Next week — same brain, but the SAME decision made many times. That's loops. See you Friday.

★ E N D O F W E E K 2 ★